The other form of the CMD instruction, which is rarely used, omits the array notation in favor of writing the command directly:

```
CMD bin/rails s -b 0.0.0.0
```

This is known as the *Shell* form because Docker executes the command via a *command shell*, prefixing it with /bin/sh -c—so in our case, it runs /bin/sh -c bin/rails s -b 0.0.0.0. The problem is that /bin/sh -c, rather than the Rails server, is the first process inside the container; since /bin/sh -c doesn't pass signals on to its subprocesses, this would cause issues when trying to terminate the server. Generally, you can just avoid this form altogether.

OK, let's rebuild our image with our new CMD instruction, remembering to tag as the latest version of railsapp:

```
$ docker build -t railsapp .
Sending build context to Docker daemon  138.8kB
«...»
Successfully built f87ad761cd0f
Successfully tagged railsapp:latest
```

With our newly built image, we can launch the Rails server—omitting the bin/rails s -b 0.0.0.0—with just:

```
$ docker run -p 3000:3000 railsapp
```

It's important to note that the CMD instruction just provides a *default* command—you can specify a different one when you issue the docker run command. For example, to list our Rake tasks, we'd run:

```
$ docker run --rm railsapp bin/rails -T
```

Note the use of --rm to delete the container after it runs. We used it here and not when running the Rails server, because this container has served its purpose after it has generated the Rake task output, whereas a container to run the Rails server can be reused.

Ignoring Unnecessary Files

You may remember that there's a separation between the Docker CLI that we use to run commands, and the Docker daemon that does most of the actual work (as we saw in the architecture diagrams for Docker for Linux on page 14 and Docker for Mac/Windows on page 14). Building an image is no different—it's the Docker daemon that actually builds the image.

How does this work in practice?